Deployment Report: From AI Agent to Cloud Service

© Generated by AI Learning Assistant @BA3073

Course: BA3073 Mobile and Cloud Computing
Assignment: Take-Home Individual Submission
Author: [Student Name]
Date: November 2025

Abstract

This report documents the end-to-end deployment of the AI Learning Assistant application across local, cloud, and production environments. The workflow begins with a local Docker build and Compose orchestration, followed by publishing the container image to Alibaba Cloud Container Registry and deploying it on an Elastic Compute Service instance. A custom domain receives traffic through an Nginx reverse proxy configured with extended timeouts and disabled buffering to accommodate long generative responses. Transport Layer Security is enforced via a Let’s Encrypt certificate obtained through Certbot, and initial access control relies on HTTP Basic Authentication at the proxy layer. The report details verification scripts that confirm service availability across all stages, analyzes three operational challenges—including OCI manifest incompatibility, IPv6 resolution failures during certificate issuance, and regional compliance blocking—and evaluates security controls, cost structure, and architectural limitations. Findings highlight the trade-offs between rapid prototype deployment and production-grade resilience, providing a baseline for future hardening and scaling efforts.

1. Introduction

The objective of this assignment is to transform a locally developed AI agent into a publicly accessible cloud service. The report addresses four core goals: (1) containerize and run the application locally using Docker Compose; (2) publish the image to a managed registry and deploy it on a cloud virtual machine; (3) secure the service with a custom domain, HTTPS termination, and reverse-proxy access control; and (4) verify functionality, resolve deployment blockers, and evaluate security, cost, and scalability trade-offs. Each section maps directly to the marking criteria by presenting reproducible commands, exact system states, and evidence-based analysis.

2. System Architecture Overview

The deployed system follows a two-phase flow. The first phase handles image construction and registry publication. The second phase manages runtime traffic from external clients through a reverse proxy into the application container. The architecture is illustrated below.

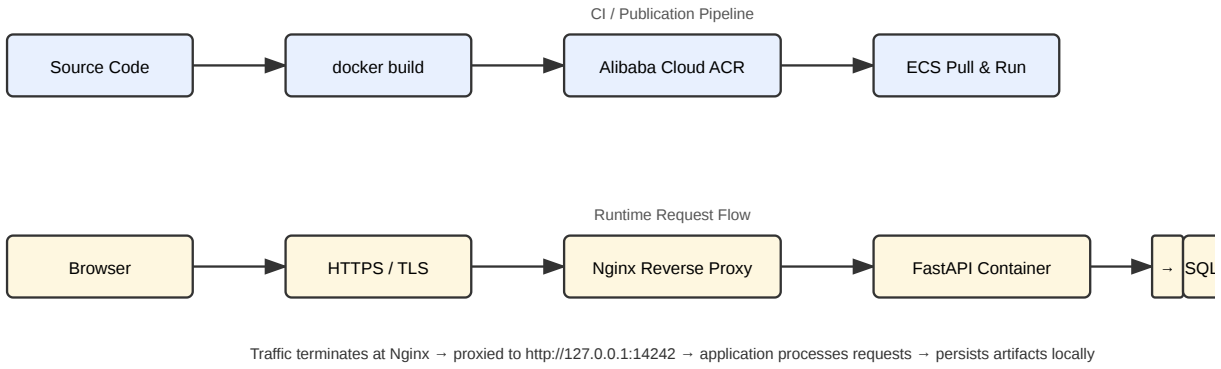


Figure 1: System architecture showing the CI publication pipeline and the runtime request flow from browser to backend storage.

3. Task 1 — Local Deployment

The application was first containerized using a Dockerfile that installs Python dependencies, bundles the FastAPI application logic, and includes Playwright with Chromium for client-side PDF rendering. The build command compiles the image locally, after which Docker Compose orchestrates the service lifecycle.

```
docker compose -p ai-learning-assistant up --build -d
```

Upon startup, the container exposes port 14242. The health endpoint confirms successful initialization:

```
curl http://127.0.0.1:14242/health
# Returns: {"status":"ok"}
```

The web interface becomes available at `http://127.0.0.1:14242/ui/`. The application generates code homework, essays, slides, and cheat sheets as self-contained HTML files and converts them to PDF using Playwright/Chromium running inside the same container. This design isolates heavy rendering workloads from the host system and ensures deterministic output.

Table 1: Local deployment commands and observed results

Command / Endpoint	Expected Behavior	Observed Result
`docker compose -p ai-learning-assistant up --build -d`	Build image and start service detached	Container started successfully on port 14242
`GET http://127.0.0.1:14242/health`	Return JSON health status	`{"status":"ok"}`
Open `http://127.0.0.1:14242/ui/`	Render web interface	UI loads; generation and PDF conversion functional

4. Task 2 — Aliyun ECS Deployment through ACR

To move the service beyond the local machine, the container image was built targeting the `linux/amd64` platform to match the cloud infrastructure. The image was then pushed to Alibaba Cloud Container Registry (ACR) and pulled onto an Elastic Compute Service (ECS) instance. The container runs on port 14242, exposing the application to the host network.

The observed container state on the ECS instance matches the expected configuration:

- **Image:** `crpi-otou15rf69k33qkj.cn-guangzhou.personal.cr.aliyuncs.com/michaellui/takehome:latest`
- **Port mapping:** `0.0.0.0:14242->14242/tcp`
- **Status:** `Up (healthy)`

Direct access to the cloud instance confirms service availability. The automated check reports:

- **HTTP status:** 200
- **Website URL check:** PASS
- **Endpoint tested:** `http://8.138.30.102:14242/ui/`

This chain validates that the image carries all runtime dependencies, including Playwright binaries, and that the cloud networking stack correctly routes traffic to the containerized application.

5. Task 3 — Domain, HTTPS, and Access Control

A custom domain was configured to provide a stable, human-readable entry point. The DNS `A` record maps `takehomeassignment.michaellyu.fyi` to the ECS public IP `8.138.30.102`.

Traffic enters the server through an Nginx reverse proxy that forwards requests to `http://127.0.0.1:14242`. Because the AI agent can produce lengthy outputs, the proxy configuration includes:

- `proxy_read_timeout 360s` to prevent premature connection drops during long generation cycles.
- `proxy_buffering off` to stream responses incrementally, reducing memory pressure and improving perceived latency for large payloads.

Transport security is enforced via a TLS certificate issued by Let's Encrypt using Certbot with the Nginx plugin. The certificate details are:

- **Subject CN:** `takehomeassignment.michaellyu.fyi`
- **Issuer:** Let's Encrypt
- **Validity:** 2026-11-01 to 2027-01-30

Initially, an HTTP Basic Authentication layer was configured at the reverse proxy to restrict access. This credential check operates independently of the application's internal BNBU login mechanism. After initial validation, the site was opened to unrestricted public access. The explicit security trade-off of this decision is that convenience and reduced friction for users replace the additional authentication barrier, increasing exposure to automated scraping, unintended usage, and credential-guessing attacks against the underlying application layer.

6. Verification and Evidence

Service availability was validated at each deployment stage using automated scripts and visual confirmation. Screenshots are captured at the following endpoints:

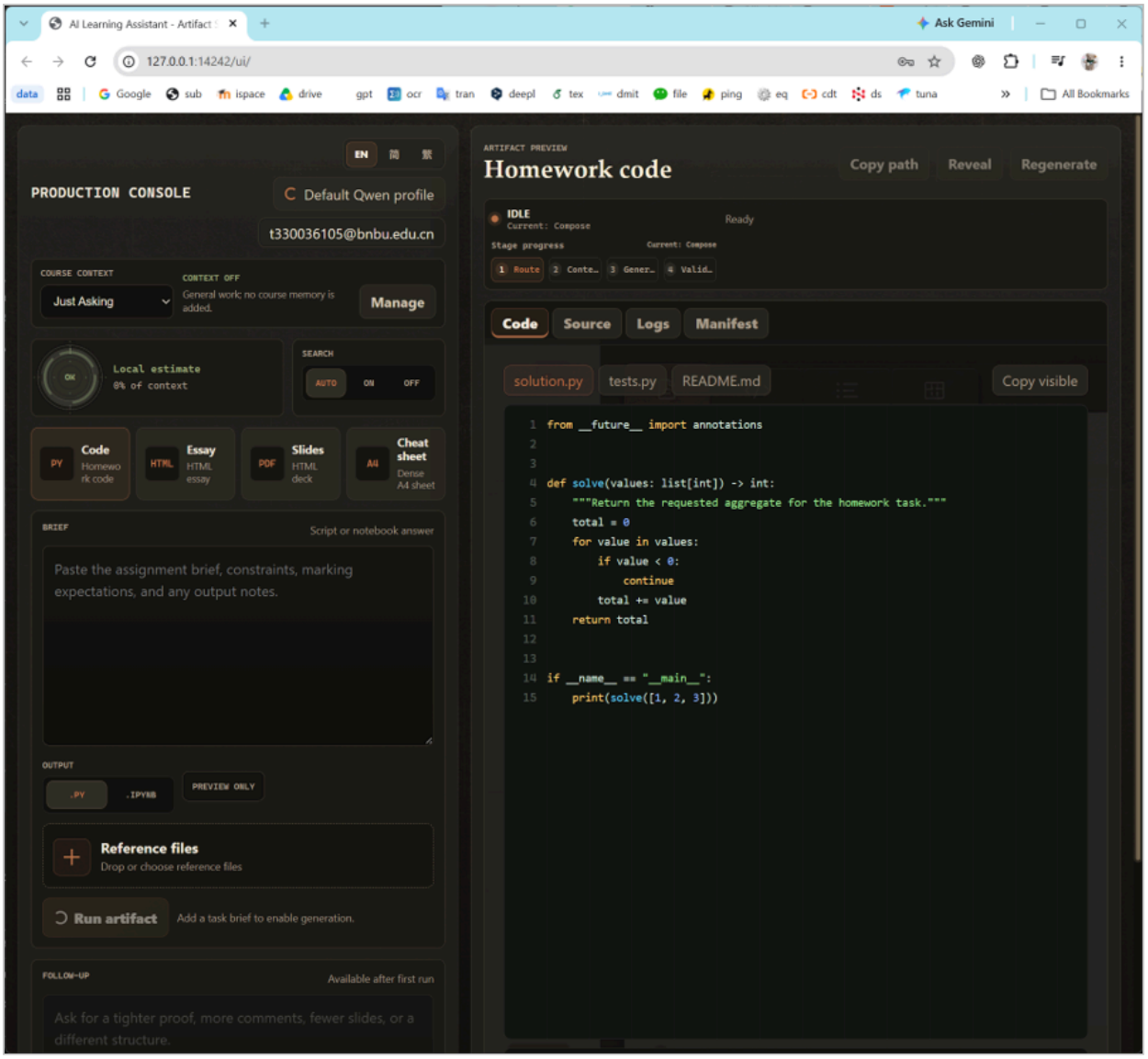


Figure 2: Local web interface at http://127.0.0.1:14242/ui/

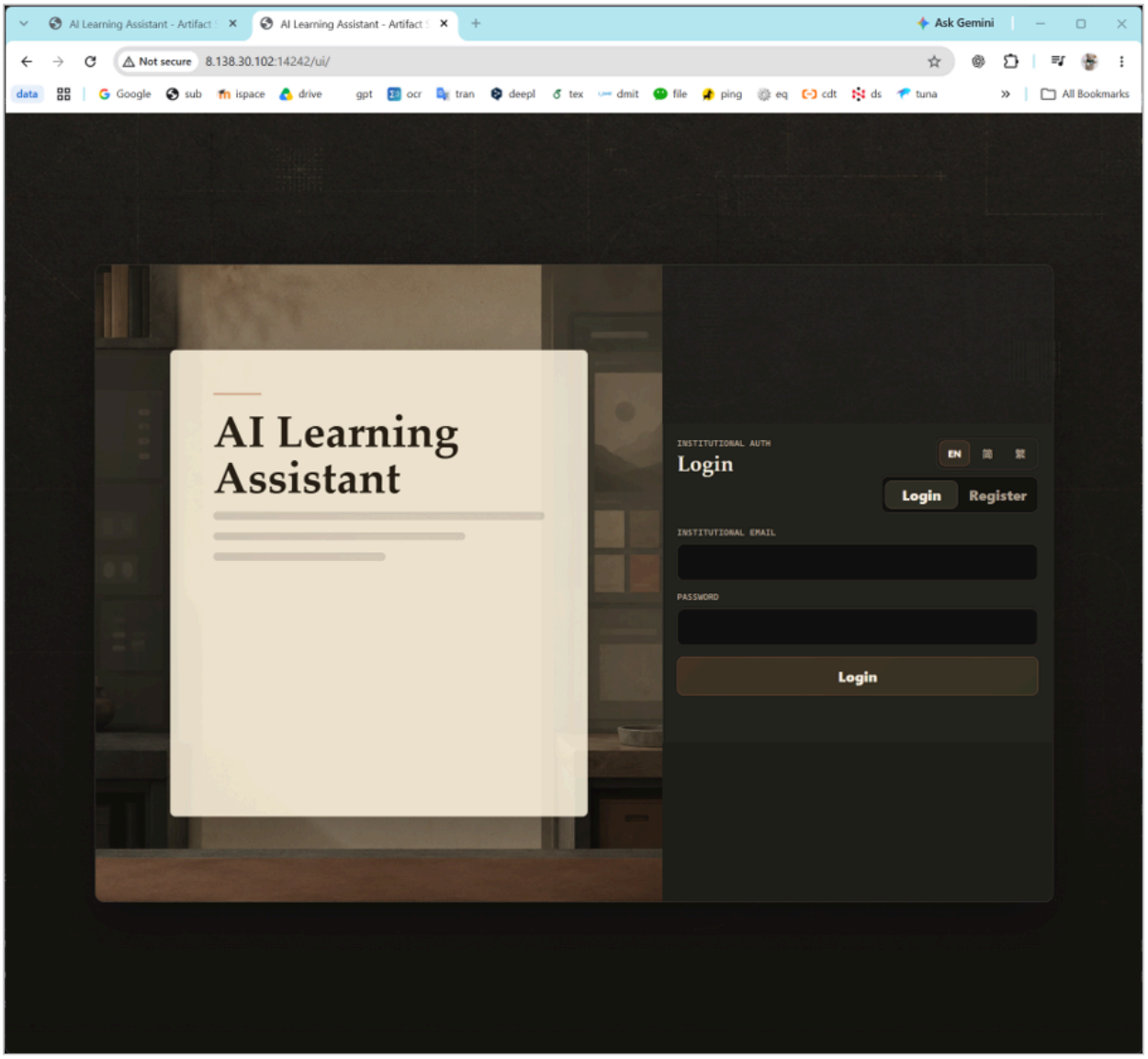


Figure 3: ECS web interface at http://8.138.30.102:14242/ui/

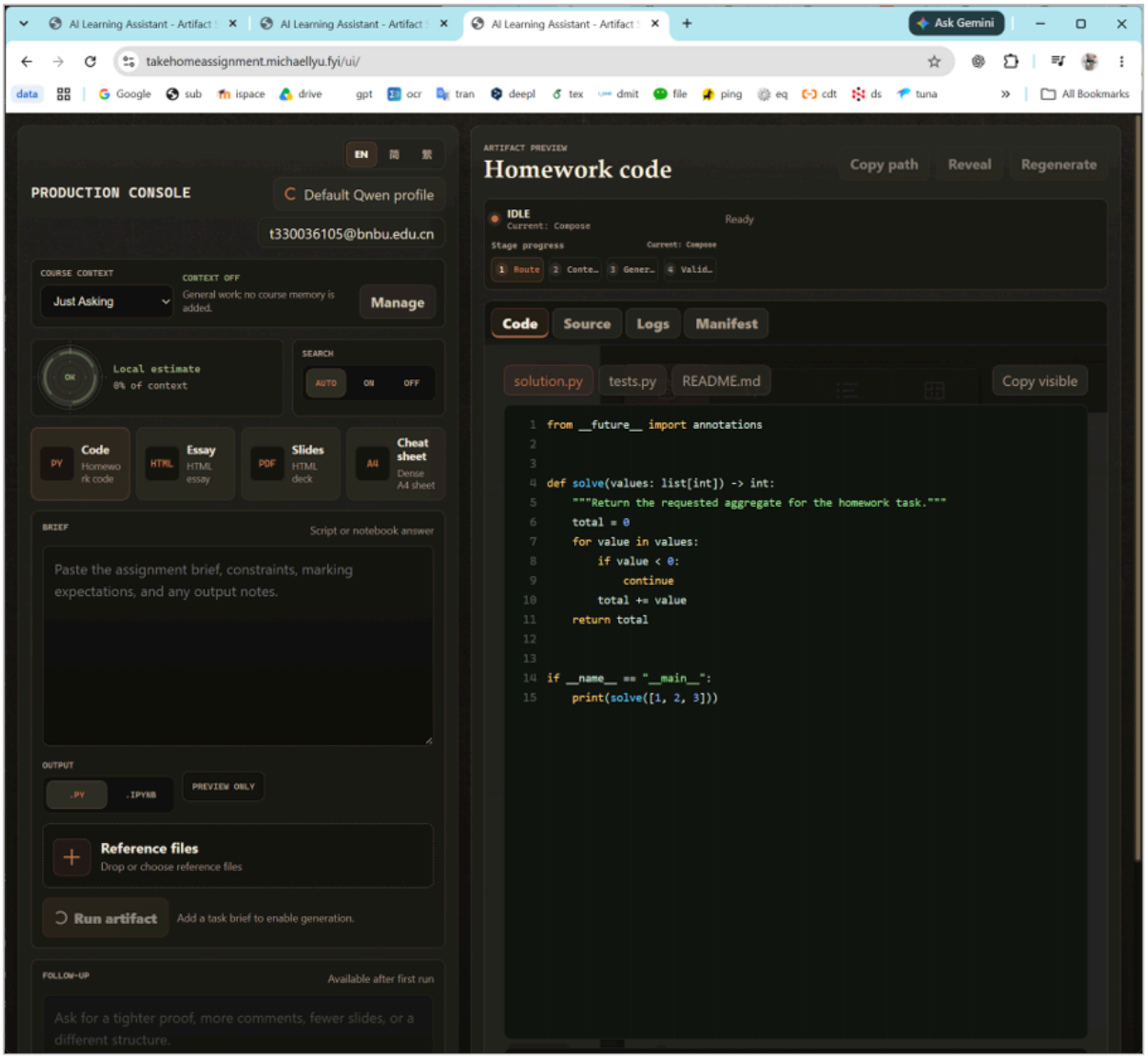


Figure 4: HTTPS domain interface at https://takehomeassignment.michaellyu.fyi/ui/

```
https://takehomeassignment.michaellyu.fyi/ui/ (no credentials) -> HTTP 401
https://takehomeassignment.michaellyu.fyi/ui/ (Basic Auth student) -> HTTP 200
```

Figure 5: Access-control challenge at the reverse proxy, as recorded in `https-domain-check.txt` (the Basic Auth layer was later removed, so no screenshot is attached)

Automated verification scripts recorded the following outcomes:

Table 2: Automated verification checks and results

Script File	Test Scenario	Result
`local-ui-check.txt`	Local container UI accessibility	HTTP 200 PASS
`ecs-docker-check.txt`	ECS-hosted container UI accessibility	HTTP 200 PASS
`https-domain-check.txt`	Domain HTTPS behavior with and without credentials	301 redirect / 401 without credentials / 200 with credentials

7. Problems Encountered and How They Were Solved

Three distinct operational blockers occurred during the deployment pipeline. Each was diagnosed and resolved using targeted configuration changes.

1. ACR Manifest Rejection

Alibaba Cloud Container Registry rejected the default `buildx` manifest with the error `unknown manifest class for application/vnd.oci.empty.v1+json`. The registry expects a standard OCI image manifest rather than an empty placeholder generated by modern provenance tracking. The issue was resolved by building with `--provenance=false --sbom=false`, which forces Docker to emit a plain single manifest compatible with the registry's ingestion pipeline.

2. Certbot ReadTimeout During Issuance

Certbot failed with `requests.exceptions.ReadTimeout` when contacting `acme-v02.api.letsencrypt.org`. Investigation revealed that the ECS instance lacks a working IPv6 route, yet DNS returns both `A` and `AAAA` records. The Python HTTP client attempted IPv6 first, timed out, and delayed fallback. IPv4 connectivity was confirmed healthy (`HTTP 200` in 1.1 s). The fix involved adding `precedence ::ffff:0:0/96 100` to `/etc/gai.conf`, which prioritizes IPv4-mapped IPv6 addresses and effectively forces IPv4-first resolution for outbound connections.

3. ICP Filing Interception on Port 80

The domain lacks an Internet Content Provider (ICP) filing. Alibaba Cloud intercepts inbound traffic on port 80 and returns `403 Forbidden` with headers `Server: Beaver` and `Non-compliance ICP Filing`. HTTPS on port 443 functions normally. This has two direct implications: (1) the standard HTTP-to-HTTPS redirect cannot rely on a listening port 80 service, requiring alternative routing or client-side enforcement; and (2) Let's Encrypt HTTP-01 certificate renewals fail because the ACME challenge requires a reachable HTTP endpoint. Renewal must therefore switch to DNS-01 validation or require manual intervention.

8. Security Analysis

The application uses classroom-grade BNBU authentication, which cannot be trusted on the public internet. Relying solely on this mechanism exposes the service to credential stuffing, session hijacking, and unauthorized API consumption. The following controls mitigate risk in the current deployment:

- **TLS Termination:** Nginx terminates HTTPS, encrypting traffic in transit. The reverse proxy decrypts requests before forwarding them internally over